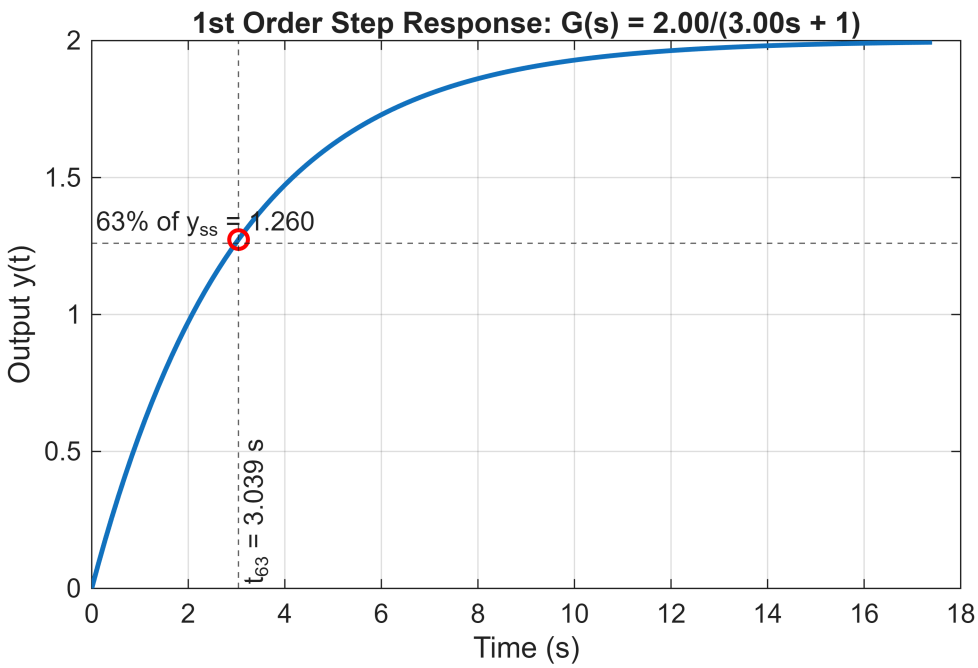
```matlab
%% Control Lab: Work1–Work3
% Date: 2026-02-27

clear; close all; clc;

%% Work1 — Transient response of a 1st order system
% Create a 1st order transfer function in MATLAB
% Construct a step response
% Find time when output reaches 63% of steady-state value (≈ t = T)
% Obtain graphs

% Choose parameters
K = 2;          % DC gain
T = 3;          % time constant (seconds)

G1 = tf(K, [T 1]);   % G(s) = K/(Ts+1)

% Step response
figure('Name','Work1: 1st Order Step Response');
[y1,t1] = step(G1);
plot(t1, y1, 'LineWidth', 1.8); grid on;
title(sprintf('1st Order Step Response: G(s) = %.2f/(%.2fs + 1)', K, T));
xlabel('Time (s)');
ylabel('Output y(t)');

% Steady-state value (final value)
y_ss = dcgain(G1);  % equals K for this system

% Find time when y reaches 63% of steady-state
target = 0.63 * y_ss;

% Find first index where response crosses target
idx = find(y1 >= target, 1, 'first');
t63 = t1(idx);
y63 = y1(idx);

hold on;
yline(target, '--', sprintf('63%% of y_{ss} = %.3f', target),
'LabelHorizontalAlignment','left');
xline(t63, '--', sprintf('t_{63} = %.3f s', t63),
'LabelVerticalAlignment','bottom');
plot(t63, y63, 'ro', 'MarkerSize', 7, 'LineWidth', 1.5);
hold off;
```




```
fprintf('Work1:\n');
```

```
Work1:
```

```
fprintf('  Time constant T = %.4f s\n', T);
```

```
  Time constant T = 3.0000 s
```

```
fprintf('  Steady-state y_ss = %.4f\n', y_ss);
```

```
  Steady-state y_ss = 2.0000
```

```
fprintf('  63%% target = %.4f\n', target);
```

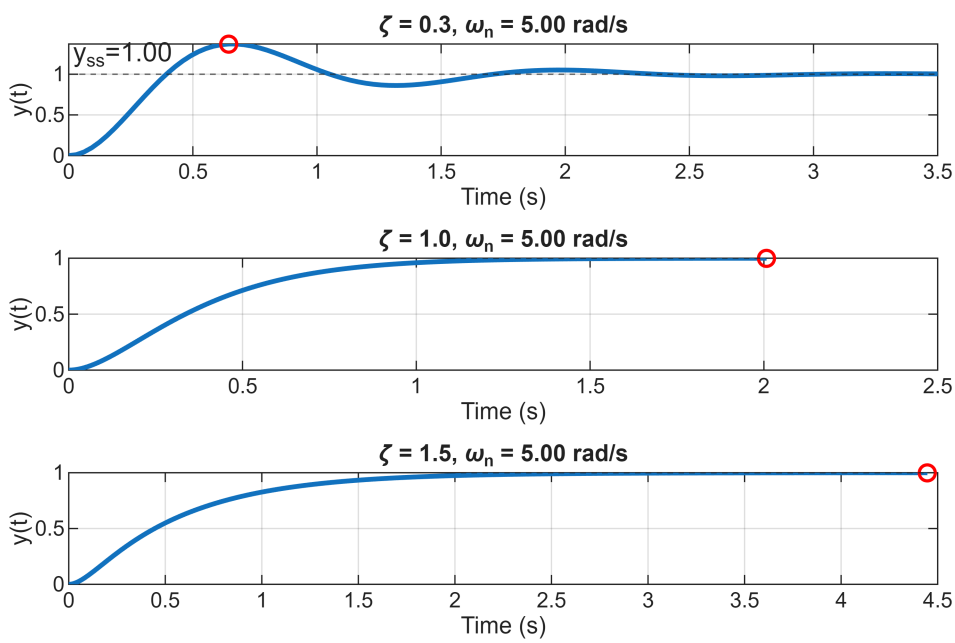
```
  63% target = 1.2600
```

```
fprintf('  Measured t_63 = %.4f s (should be close to T)\n\n', t63);
```

```
  Measured t_63 = 3.0394 s (should be close to T)
```

```
%% Work2 — 2nd order system: comparison of damping effects
% Standard 2nd order system:
% G(s) = wn^2 / (s^2 + 2*zeta*wn*s + wn^2)
% Construct step-response for zeta = 0.3, 1, 1.5
% Output overshoot for each case

wn = 5;  % natural frequency (rad/s)
zetas = [0.3 1 1.5];

figure('Name','Work2: 2nd Order Step Responses');
tiledlayout(3,1, "TileSpacing","compact");
```

```
OS = zeros(size(zetas));

for i = 1:numel(zetas)
    zeta = zetas(i);
    G2 = tf(wn^2, [1 2*zeta*wn wn^2]);

    nexttile;
    [y2,t2] = step(G2);
    plot(t2, y2, 'LineWidth', 1.8); grid on;
    title(sprintf('\\zeta = %.1f, \\omega_n = %.2f rad/s', zeta, wn));
    xlabel('Time (s)'); ylabel('y(t)');

    % Overshoot definition: (max - steady)/steady * 100%
    yss2 = dcgain(G2);          % should be 1 for this standard form
    ymax = max(y2);
    OS(i) = max(0, (ymax - yss2)/yss2 * 100); % clamp to 0 for no overshoot

    hold on;
    yline(yss2, '--', sprintf('y_{ss}=%.2f', yss2),
'LabelHorizontalAlignment','left');
    plot(t2(y2==ymax), ymax, 'ro', 'MarkerSize', 6, 'LineWidth', 1.2);
    hold off;
end
```




```
fprintf('Work2 Overshoot Results (Step Response):\n');
```

```
Work2 Overshoot Results (Step Response):
```

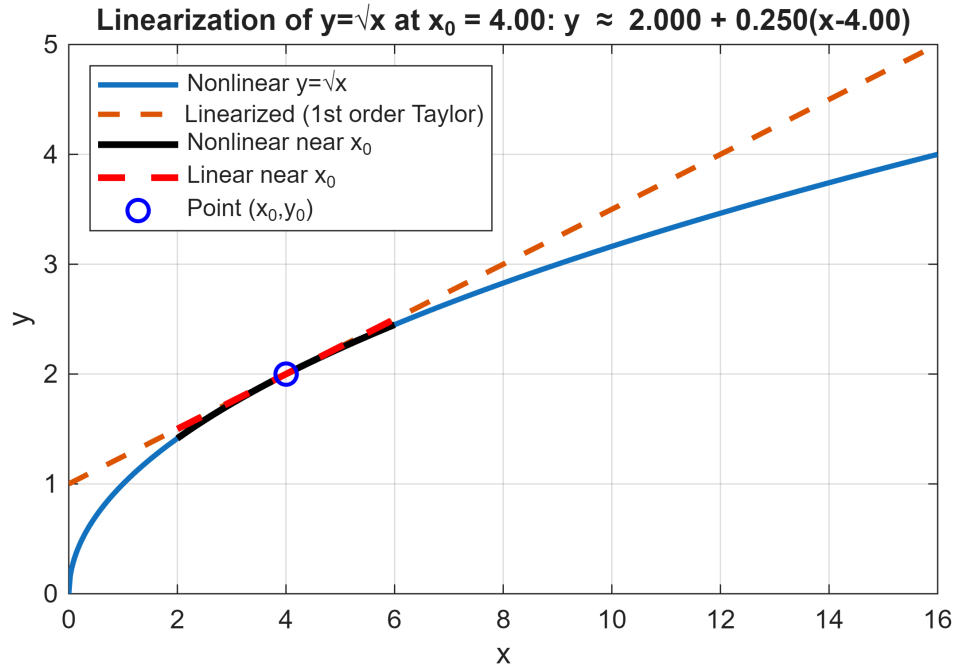
```
for i = 1:numel(zetas)
    fprintf('  zeta = %.1f -> Overshoot = %.2f %%\n', zetas(i), OS(i));
```

```
end
```

```
zeta = 0.3 -> Overshoot = 37.14 %
zeta = 1.0 -> Overshoot = 0.00 %
zeta = 1.5 -> Overshoot = 0.00 %
```

```
fprintf('\n');
```

```
%% Work3 — Linearization of nonlinear dependence (1st order Taylor)
% Nonlinear function: y = sqrt(x)
% Select any X0
% Obtain graph
% Show near X0 the linear model is close to nonlinear

% Choose linearization point
x0 = 4;   % you can change this
y0 = sqrt(x0);

% First derivative: dy/dx = 1/(2*sqrt(x))
m = 1/(2*sqrt(x0));

% Define functions
f = @(x) sqrt(x);
f_lin = @(x) y0 + m*(x - x0);

% Domain (avoid x<0 for sqrt)
x = linspace(0, 16, 600);

figure('Name','Work3: Taylor Linearization of sqrt(x)');
plot(x, f(x), 'LineWidth', 2); grid on; hold on;
plot(x, f_lin(x), '--', 'LineWidth', 2);

% Highlight neighborhood around x0
x_zoom = linspace(max(0,x0-2), x0+2, 200);
plot(x_zoom, f(x_zoom), 'k', 'LineWidth', 2.5);           % emphasize true curve
near x0
plot(x_zoom, f_lin(x_zoom), 'r--', 'LineWidth', 2.5);     % emphasize linear near x0

plot(x0, y0, 'bo', 'MarkerSize', 8, 'LineWidth', 1.5);
legend('Nonlinear y=\surdx','Linearized (1st order Taylor)','Nonlinear near
x_0','Linear near x_0', 'Point (x_0,y_0)', ...
       'Location','northwest');
title(sprintf('Linearization of y=\\surdx at x_0 = %.2f: y \\approx %.3f + %.3f(x-
%.2f)', x0, y0, m, x0));
xlabel('x'); ylabel('y');
```




```
fprintf('Work3:\n');
```

```
Work3:
```

```
fprintf('  x0 = %.4f\n', x0);
```

```
  x0 = 4.0000
```

```
fprintf('  y0 = sqrt(x0) = %.4f\n', y0);
```

```
  y0 = sqrt(x0) = 2.0000
```

```
fprintf('  slope m = 1/(2*sqrt(x0)) = %.4f\n', m);
```

```
  slope m = 1/(2*sqrt(x0)) = 0.2500
```

```
fprintf('  Linear model: y ≈ %.4f + %.4f (x - %.4f)\n', y0, m, x0);
```

```
  Linear model: y ≈ 2.0000 + 0.2500 (x - 4.0000)
```